

Claude e Claude Code in pratica

Dal prompting agli agenti

Simone Scannapieco

Corso base per Tecnodata Trentina S.r.l., Trento, Italia

Modulo 4 — Un agente nel repository

Il ciclo degli strumenti

Gli strumenti di Claude Code

Che cosa può fare, e che cosa chiede prima di farlo

Nel Modulo 4, un agente al lavoro in un repository: Claude Code. Per Anthropic gli **agenti** sono, in genere, modelli che usano degli **strumenti** in un ciclo, guidati da ciò che l'ambiente restituisce

Sono gli strumenti a farne un agente: senza, Claude può solo **rispondere con del testo**. Tra le famiglie: i **file**, da leggere e modificare; la **ricerca**; l'**esecuzione** di comandi. A Claude servono gli **stessi strumenti** che i programmatori usano ogni giorno

strumento	che cosa fa	chiede il permesso?
Read	legge il contenuto di un file	**no**
Edit	fa modifiche mirate a un file	sì
Write	crea o sovrascrive un file	sì
NotebookEdit	modifica le celle di un notebook Jupyter	sì
Bash	esegue comandi di shell	sì
WebFetch	legge il contenuto di un indirizzo dato	sì
WebSearch	fa ricerche sul web	sì
Agent	avvia un subagent, con una sua finestra di contesto	avviarlo **no**; le chiamate del subagent passano dalle regole

È la tabella della documentazione, per la modalità Manual e i file nella cartella di lavoro: che cosa parte davvero senza chiedere dipende dalla **modalità** e dalle **regole**, nella parte «Permessi e scala della fiducia»

Cercare, su Linux e WSL

La ricerca passa da Bash; e prima di modificare, si legge

- Su macOS, Linux e WSL, **Glob** e **Grep** restano fuori dagli strumenti predefiniti: Claude cerca con `find` e `grep` **dentro Bash**, che nella sua shell sono versioni integrate di `bfs` e `ugrep`
- Sono comandi di **sola lettura**, come `ls`, `cat` e `head`: di regola **non chiedono** il permesso
- Glob e Grep tornano se li nominate all'avvio, in `--tools` o in `--allowedTools`; nominarne uno in `--allowedTools` li riporta **tutti e due**
- **Prima si legge**: Claude legge il file nella conversazione prima di modificarlo. I modelli più vecchi lo richiedono sempre; i più recenti possono modificare un file non letto, se leggerlo non chiederebbe un permesso e Read è disponibile
- Con Edit, il testo da sostituire deve comparire nel file **esattamente** com'è scritto, e **una volta sola**

M4-D03 Un giro del ciclo, registrato

Demo del docente in Claude Code: un giro, richiesta per richiesta

- Un compito piccolo, con un controllo che Claude può eseguire da sé: nel progetto delle urgenze la regola dice «almeno 7 giorni», il codice > 7, e un test è **rosso**. Guardate le **richieste** al modello: che strumenti chiama ognuna, e quanti **token in entrata** porta
- Dalla pagina sui costi di Claude Code: «Claude Code sends your full conversation with every request, and each time Claude uses tools it sends another request carrying that batch of tool results.»

`prompt.txt, il compito: lo stesso testo in ogni esecuzione, a capo per la slide`

```
<instructions>
Il confine dei 7 giorni in urgenza.py non è rispettato:
correggilo, poi esegui `python3 -m unittest` e continua finché
i test passano.
</instructions>

<constraints>
Esegui un comando alla volta.
</constraints>
```

M4-D03 Un giro del ciclo, registrato

Demo del docente in Claude Code: un giro, richiesta per richiesta

- `claude -p` parte da una **copia nuova** del progetto, su Sonnet con effort low, con gli strumenti consentiti Read, Edit e `Bash(python3 -m unittest*)`; lo `stream-json` registra ogni evento
- **Indovinate, a voce**: per correggere una riga e far passare i test, quante richieste farà Claude Code?
- **Nella registrazione del corso:** **4 richieste al modello** e **3 chiamate a strumenti**, in ognuna delle tre esecuzioni

M4-D03 Un giro del ciclo, registrato

Demo del docente in Claude Code: un giro, richiesta per richiesta

La registrazione del corso: tre esecuzioni, ognuna in una copia nuova del progetto; poi il controllo dell'elemento, che legge lo stream di ognuna e riesegue i test nella copia

richiesta	in entrata	aumento	strumento
1	18.075 token		Read urgenza.py
2	18.308 token	+233	Edit urgenza.py
3	18.545 token	+237	Bash «python3 -m unittest 2>&1 | tail -20»
4	18.647 token	+102	nessuno strumento

Il primo giro, le sue quattro richieste: le righe del controllo, in tabella
Registrazione del corso, 12 novembre 2026, Claude Code 2.1.322, Sonnet

esecuzione	ogni richiesta rimanda tutta la conversazione
giro-1	**confermata**: l'entrata non scende mai, da 18.075 a 18.647 token in 4 richieste
giro-2	**confermata**: l'entrata non scende mai, da 18.075 a 18.595 token in 4 richieste
giro-3	**confermata**: l'entrata non scende mai, da 18.075 a 18.647 token in 4 richieste

Il controllo decide la prima metà della frase: **confermata** in tutte e tre, **3 su 3**; e i test, rieseguiti ora nella copia, sono **verdi (3 test, OK)** in tutte e tre

Il contesto si riempie

Che cosa c’è dentro, come si guarda, e quando si svuota

- La **finestra di contesto** contiene tutto ciò che Claude sa della sessione, anche ciò che nel terminale **non compare**
- Prima ancora che scriviate entrano i `CLAUDE.md`, la memoria automatica (auto memory), i nomi degli strumenti dei server MCP e le descrizioni delle skill. `CLAUDE.md` e memoria sono del Modulo 6, le skill del Modulo 5
- `/context` mostra dal vivo quanto occupa ogni categoria, con dei suggerimenti, e quali `CLAUDE.md` e file di memoria sono caricati
- Lavorando, il contesto si riempie, e Claude lo **compatta** da solo, con un riassunto; ma le istruzioni date all’inizio **possono andare perse**
- Ogni **sessione nuova** parte con una finestra nuova, senza la cronologia delle sessioni precedenti

Un controllo che risponde sì o no

Perché Claude sappia quando ha finito

- Claude si ferma quando il lavoro **sembra** finito. Se non ha nulla da eseguire che gli dica se lo è davvero, «sembra finito» è **tutto ciò che ha**
- Dategli qualcosa che risponde **«passa» o «fallisce»**, e Claude può chiudere il ciclo da solo: fa la modifica, esegue il controllo e, se qualcosa si è rotto, legge l'uscita e **corregge** prima di rieseguirlo
- Il controllo può essere quasi qualsiasi cosa, purché Claude ne possa **leggere il risultato**. Nella demo erano i test, `python3 -m unittest`, e il prompt li nominava
- Se il repository ne ha già uno, la **riga più utile** di un `CLAUDE.md` è il comando che lo esegue: il `CLAUDE.md` è del Modulo 6

Ogni risultato entra nel contesto e viaggia a ogni richiesta; un controllo che risponde sì o no gli dice quando ha finito.

Quanto costa un giro

Ogni richiesta rimanda tutto: anche ciò che non serve più

- Ogni richiesta rimanda **tutta la conversazione**: una sessione che ha accumulato tre compiti scollegati li **rilegge tutti e tre** a ogni turno del quarto
- Nella prima esecuzione registrata di M4-D03, giro-1, l'entrata è salita da 18.075 a 18.647 token in quattro richieste: ogni richiesta **pesa più** della precedente
- Fra un compito e l'altro, `/clear`: la conversazione vecchia toglie spazio ai file che servono dopo, e costa token a **ogni messaggio**. Un contesto concentrato dà risposte migliori, e costa meno